supports the ability to specify which configuration file is used. The following sections cover the configuration files in more detail, but let's start by using the default:

```
import com.typesafe.config.ConfigFactory
import com.typesafe.config.Config

val config: Config = ConfigFactory.load()
```

7.1.1 Order

When using the default, the library tries to find the configuration file. Since the library supports several different configuration formats, it looks for different files in the following order:

- *application.properties*—This file should contain the configuration properties in the Java property file format.
- *application.json*—This file should contain the configuration properties in the JSON style.
- *application.conf*—This file should contain the configuration properties in the Human-Optimized Config Object Notation (HOCON) format. This format is based on JSON but is easier to read. (You can find more details on HOCON and the Typesafe Config library at https://github.com/typesafehub/config.)

It's possible to use all the different files in your project at the same time. Here's an example of an application.conf file:

```
MyAppl {
  version = 10
  description = "My application"
  database {                                    ❶
    connect="jdbc:mysql://localhost/mydata"
    user="me"
```